

shreyas@Grigori: ~

shreyas@Grigori:~\$ tree

```
.
├── DIRECTORY...
├── OSSP
│   ├── LICENSE
│   ├── Makefile
│   ├── README.md
│   ├── assets
│   │   └── architecture.png
│   ├── bin
│   ├── docs
│   │   ├── design.md
│   │   └── report.pdf
│   ├── include
│   │   ├── builtin.h
│   │   ├── executor.h
│   │   ├── parser.h
│   │   └── shell.h
│   ├── obj
│   ├── scripts
│   │   └── run.sh
│   ├── src
│   │   ├── builtin.c
│   │   ├── executor.c
│   │   ├── main.c
│   │   ├── parser.c
│   │   └── shell.c
│   └── tests
│       ├── test_executor.c
│       └── test_parser.c
```

shreyas@Gngon: ~

```
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
├── tests
│   ├── test_executor.c
│   └── test_parser.c
├── [OSLAB]...
├── file1.c
├── file1.txt
└── hello.c
```

2 directories, 21 files

shreyas@Grigori: ~

GNU nano 7.2fork_example.c

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid;
    pid = fork();    // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

[Read 22 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/ Go To Line	M-E Redo	M-6 Copy